

Data Structures Assignment 2

Alua Rakhimzhanova SE-2509

DynamicArray and MinHeap use int arrays that double when full. MyLinkedList uses nodes with next links and a tail pointer.

Complexity

n is the size, C is the capacity. Average index costs assume equally likely valid indices. Searches may find a value at any position or miss it.

Operation	Best	Average	Worst	Extra space	Reason
Array add(x)	$\Theta(1)$	$\Theta(1)$ spare $\Theta(n)$ full	$\Theta(n)$	$\Theta(n)$ growth $\Theta(1)$ otherwise	One write, or copy n values to grow.
Array add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$ growth $\Theta(1)$ otherwise	Shift $n-i$ values; grow if full.
Array remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Shift $n-i-1$ values left.
Array get(i)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Read one cell directly.
Array contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Scan until a match or the end.
List add(x)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Add one node using tail.
List add(i,x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Head and tail are direct; otherwise walk.
List remove(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Find the previous node, except at head.
List get(i)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Follow i links from head.
List contains(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Check nodes until a match or null.
Heap insert(x)	$\Theta(1)$	$O(\log n)$ spare $\Theta(n)$ full	$\Theta(n)$	$\Theta(n)$ growth $\Theta(1)$ otherwise	Move up at most $\log n$ levels; grow if full.
Heap peekMin()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Read the root.
Heap extractMin()	$\Theta(1)$	$\Theta(\log n)^*$	$\Theta(\log n)$	$\Theta(1)$	Move down until heap order is restored.

Spare means a free slot. Full means $n = C$. Heap insertion without growth is between $\Omega(1)$ and $O(\log n)$, depending on the values. Array append is $\Theta(1)$ amortized: growth is expensive, but happens rarely. Heap insertion is $O(\log n)$ amortized.

* Average extraction assumes distinct random values. Equal values can give $\Theta(1)$. Arrays use $\Theta(C)$ storage and do not shrink; the list uses $\Theta(n)$. `size()`, `metrics()` and `reset()` take $\Theta(1)$. The test snapshot takes $\Theta(C)$ time and space.

Counters: a step is one array read or one next-link advance, including null. A move is an existing value copied or a head, tail or next update. Growth copies count; a heap swap counts twice. Only value comparisons count. New values and local pointer assignments do not count as moves.

Loop invariants

DynamicArray contains

Invariant: Before index i , no checked value equals x . The array stays unchanged.

Initialization: $i = 0$, so no values have been checked.

Maintenance: If $\text{data}[i]$ equals x , return true. Otherwise, move to $i+1$; all earlier values still do not match.

Termination: A match gives true. Reaching size means no match, so return false.

Conclusion: The result is true exactly when x is present.

DynamicArray remove

Let A be the original array, n its size and k the removal index. Save $A[k]$ first.

Invariant: Before iteration i , values before k are unchanged. For $k \leq j < i$, $\text{data}[j] = A[j+1]$. From i to $n-1$, values are still unchanged.

Initialization: $i = k$, so no values have moved.

Maintenance: Copy $\text{data}[i+1]$ to $\text{data}[i]$, then increase i . One more value is in its correct position.

Termination: At $i = n-1$, all required values have shifted left. Decrease size by one.

Conclusion: The array has its original order with $A[k]$ removed. Return the saved $A[k]$.

Benchmark

Sizes: 100, 1,000, 10,000 and 100,000. Both lists use the same data from `Random(42)`. Each case has 20 warm-ups and 5 measured runs; the middle time is saved. W1-W3 exclude initial filling. W4 includes insertions. Result checks are outside the timer. All 10 JUnit tests passed.

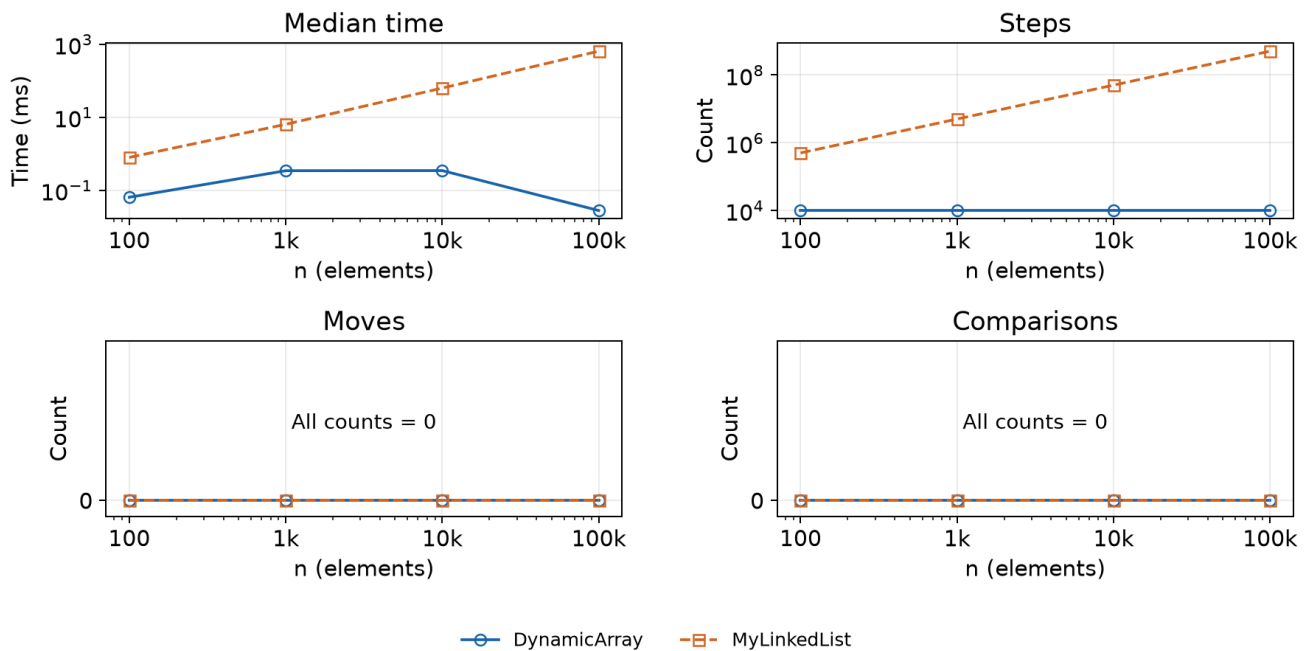
Discussion

Array get reaches an index directly. List get walks from the head. Nearby array values share CPU cache lines. This spatial locality also makes scanning faster. List nodes can be far apart in memory. Following their links is called pointer chasing. So equal Big-O or similar step counts can still give different times. Nodes also need object headers, links and padding. Creating and removing nodes adds work for the garbage collector. The list is useful for frequent insertions and removals at the head. Middle operations still need traversal. The heap suits jobs processed by smallest priority first.

Random access and search

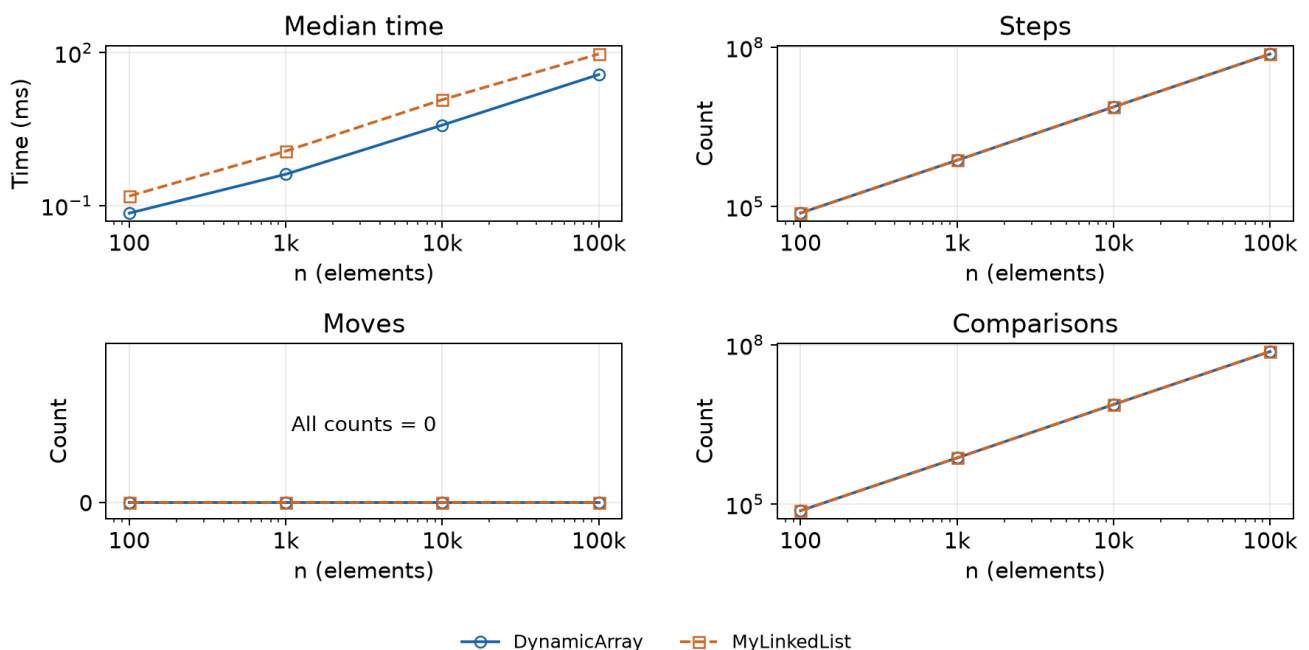
Nonzero graphs use log scales. Zero counts use a normal scale. Equal lines overlap.

W1 - Random access



W1 uses 10,000 random get calls. The array always makes 10,000 reads. The list follows more links as n grows. At n = 100,000, times are 0.028 ms and 640.586 ms.

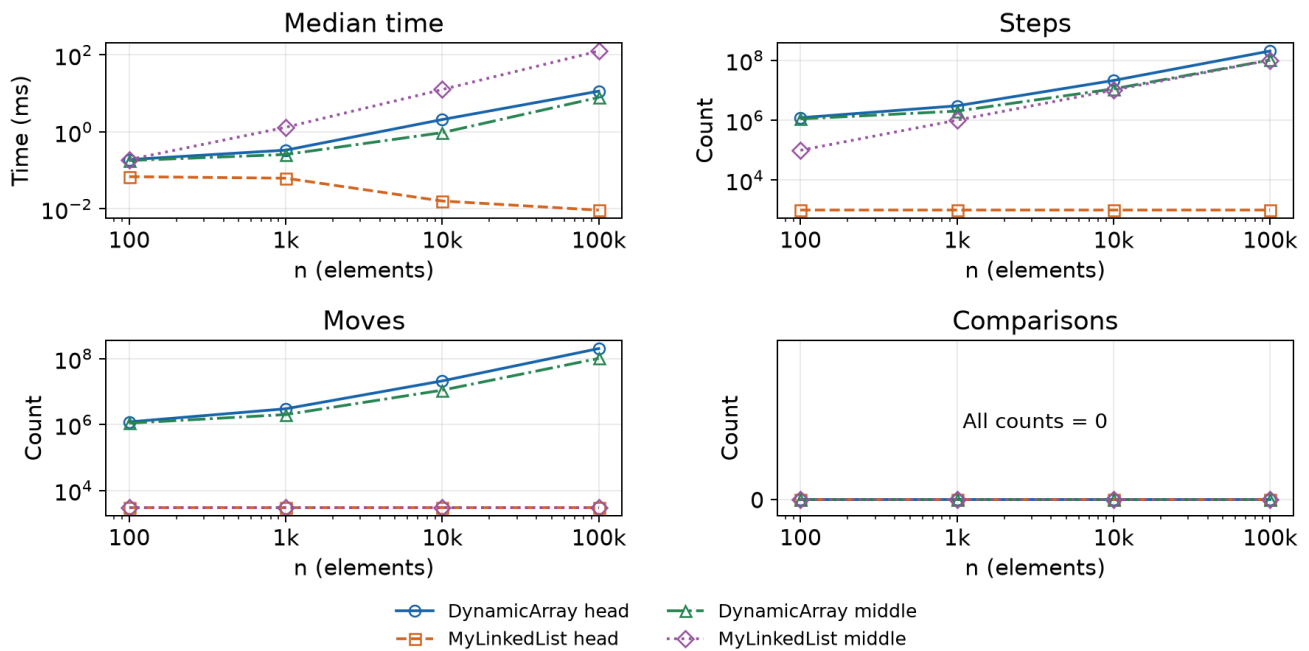
W2 - Search



W2 uses 1,000 searches: 500 present and 500 absent. At n = 100,000, both make 73,682,044 comparisons. The array takes 36.832 ms and the list 93.285 ms. Each successful list search saves one next-link step, so it has 500 fewer steps.

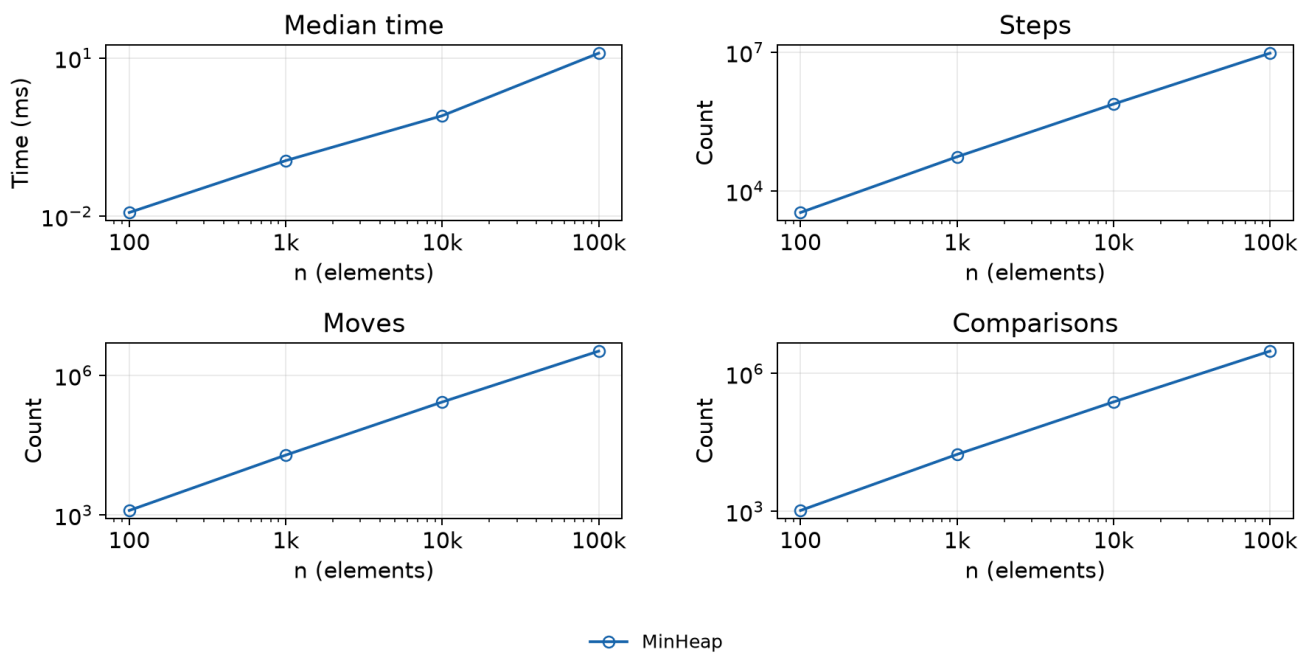
Updates and priority processing

W3 - Insert and remove



W3 adds 1,000 values, then removes 1,000. The index is 0 or the original $n/2$. At $n = 100,000$, head times are 11.385 ms for the array and 0.009 ms for the list. Both list cases make 3,000 link updates, but the middle case also needs traversal.

W4 - Priority processing



W4 inserts and extracts n heap values. At $n = 100,000$, it takes 12.318 ms and 3,059,125 comparisons. The output is sorted. Total time is $O(n \log n)$. Short timings may vary because Java and other programs affect the measurements.